

Zeitfenster, Raum, Buchung

Organisation: Sie arbeiten im Zweierteam in einem gemeinsamen Repository, das Sie das ganze Semester weiterführen. Eine Person übernimmt die Rolle A, die andere die Rolle B. Steht vor einem Arbeitsschritt (A) oder (B), erledigt ihn nur diese Person. Das Testat schreibt jede Person allein.

Abgabe: ein ZIP des Projektordners einschließlich des Ordners `.git` über Moodle/APA bis [OFFEN: Datum]. Der letzte Commit auf `main` trägt den Tag `abgabe-1`.

Sie schreiben den Java-Code dieses Blatts ohne KI-Werkzeug und bedienen Git selbst, Befehl für Befehl. Im Testat und in der Vorführung erklären Sie jede Zeile und jeden Befehl.

Freiraum ist eine Buchungsanwendung für Lern- und Laborräume. Alle Räume, Gruppen, Zeiten und Regeln in diesem Praktikum sind frei erfunden und beschreiben keine realen Räume oder Buchungsregeln der THI. Uhrzeiten rechnen Sie in Minuten ab Mitternacht (10:30 ist 630), Wochentage als Zahl von 1 (Montag) bis 5 (Freitag).

Erste Aufgabe (45 Minuten): Laden Sie `freiraum-start-1.zip` aus Moodle. Das `README.md` darin enthält die Git-Befehle für die Einrichtung; gehen Sie es in dieser Reihenfolge durch.

- Beide legen, falls noch nicht vorhanden, ein GitHub-Konto an. (A) erstellt ein **privates** Repository `freiraum` ohne `README` und lädt B ein.
- Beide stellen Namen, E-Mail-Adresse und das Verhalten von `git pull` ein:

```
git config --global user.name "Vorname Nachname"
git config --global user.email "adresse@example.org"
git config --global pull.rebase false
```

Die dritte Zeile legt fest, dass `git pull` auseinandergelaufene Stände per Merge zusammenführt. Ohne sie bricht ein frisch installiertes Git in der sechsten Aufgabe vor dem Konflikt mit `fatal: Need to specify how to reconcile divergent branches.` ab.

- (A) entpackt den Startstand, trägt in `NOTIZEN.md` beide Namen ein und lädt die Vorlage hoch (`README`, Schritt 3). (B) klonet das Repository mit `git clone`.
- Beide öffnen den Ordner mit der `pom.xml` in der Entwicklungsumgebung und starten `PruefungFormat`. Die letzte Zeile lautet `6 OK, 0 FEHLER`.
- Jede Person legt ihren Branch an: (A) `a-zeitfenster`, (B) `b-buchung`. Als ersten Commit legt (A) in `src/main/java/freiraum/` die Datei `Zeitfenster.java` an, (B) die Datei `Buchung.java`, jeweils nur mit `package freiraum;`, Klassenkopf und leerem Rumpf. Pushen Sie den Branch mit `git push -u origin <Branch>`.
- Jede Person öffnet auf GitHub einen Pull Request nach `main`. Die andere Person liest ihn und mergt. Danach holen beide den gemeinsamen Stand mit `git switch main` und `git pull`.

Zweite Aufgabe (15 Minuten): Fotografieren Sie den Belegungsplan an einer Raumtür, ohne dass Personen auf dem Bild sind, oder nehmen Sie den fiktiven Plan aus Moodle [OFFEN: Plan bereitstellen]. Das Foto geben Sie nicht ab. Notieren Sie in `NOTIZEN.md` unter „Drei Regeln, die der Belegungsplan stillschweigend voraussetzt“ drei solche Regeln als ganze Sätze. Eine Regel dieser Art lautet: „Ein Eintrag endet später, als er beginnt.“ Suchen Sie drei andere. Committen Sie auf `main` und pushen Sie.

Dritte Aufgabe (60 Minuten): Wechseln Sie in Ihren Branch und holen Sie den gemeinsamen Stand hinein: `git switch a-zeitfenster` bzw. `git switch b-buchung`, dann `git merge main`. Jetzt liegen beide leeren Klassen in Ihrem Branch. Schreiben Sie nur in Ihrer eigenen Datei.

- (A) Zeitfenster mit den privaten Attributen `tag`, `beginn` und `ende`. Der Konstruktor `Zeitfenster(int tag, int beginn, int ende)` wirft eine `IllegalArgumentException`, wenn der Tag nicht zwischen 1 und 5 liegt, der Beginn vor 07:00 (420) oder das Ende nach 22:00 (1320) liegt oder das Ende nicht nach dem Beginn liegt. Dazu kommen die Getter `getTag()`, `getBeginn()`, `getEnde()`, die Methoden `int dauerMinuten()` und `boolean ueberschneidet(Zeitfenster z)` sowie `toString()` im Format `Di 10:00–11:30` mit `Format.wochentag` und `Format.uhrzeit`. Zwei Fenster am selben Tag, von denen eines um 10:00 endet und das andere um 10:00 beginnt, überschneiden sich **nicht**.
- (B) Buchung mit den privaten Attributen `gruppe` (Text), `personen` (ganze Zahl) und `zeitfenster` (ein Zeitfenster). Der Konstruktor `Buchung(String gruppe, int personen, Zeitfenster zeitfenster)` wirft eine `IllegalArgumentException`, wenn die Gruppe null ist oder nur aus Leerzeichen besteht, weniger als eine Person angegeben ist oder das Zeitfenster fehlt. Dazu kommen die Getter `getGruppe()`, `getPersonen()`, `getZeitfenster()` und `toString()` im Format `Di 10:00–11:30 Lerngruppe Statistik, 4 Pers.` (zwei Leerzeichen nach der Uhrzeit).

Klären Sie, bevor Sie den Pull Request öffnen:

- Was brauchen Sie von der Klasse der anderen Person, damit Ihr Code kompiliert, und was erst, damit er richtig läuft?

Committen Sie in kleinen Schritten mit Nachrichten, die sagen, was sich geändert hat. Pushen Sie, öffnen Sie je einen Pull Request, lesen Sie den Code der anderen Person und mergen Sie beide nach `main`.

Vierte Aufgabe (60 Minuten): Schreiben Sie `Raum` gemeinsam an einem Rechner im Branch `raum`, den Sie von `main` aus anlegen. Nach 30 Minuten wechselt die Tastatur.

- Attribute: `kennung` (Text), `plaetze` (ganze Zahl), `Buchung[] buchungen` mit Platz für 20 Buchungen und `anzahl`. Belegt sind nur die Plätze 0 bis `anzahl - 1`.
- Der Konstruktor `Raum(String kennung, int plaetze)` weist eine leere Kennung und weniger als einen Platz mit einer `IllegalArgumentException` ab. Getter: `getKennung()`, `getPlaetze()`, `getAnzahl()`.
- `boolean istFrei(Zeitfenster z)` liefert `true`, wenn sich keine eingetragene Buchung mit `z` überschneidet. Die Methode sieht nur die belegten Plätze an.
- `boolean buchen(Buchung b)` liefert `false`, wenn alle 20 Plätze belegt sind, wenn mehr Personen kommen, als der Raum Plätze hat, oder wenn das Zeitfenster nicht frei ist. Sonst trägt die Methode die Buchung ein und liefert `true`.
- `toString()` liefert zum Beispiel `LR-1 (12 Plätze)`. `void belegungAusgeben()` druckt eine Kopfzeile und je belegtem Platz eine Zeile:

```
LR-1 (12 Plätze), 2 Buchungen
Di 10:00–11:30 Lerngruppe Statistik, 4 Pers.
Di 11:30–13:00 Team Blau, 3 Pers.
```

Tragen Sie danach in `NOTIZEN.md` die Invarianten von `Zeitfenster`, `Buchung` und `Raum` ein, je Invariante als Satz und mit der Zeile, die sie schützt. Findet sich für eine Invariante keine Zeile, schreiben Sie das dazu. Committen, pushen, Pull Request, mergen.

Fünfte Aufgabe (45 Minuten): Jede Person prüft die Klasse, die sie nicht geschrieben hat, in einem eigenen Branch (a-pruefung, b-pruefung). Vorbild ist PruefungFormat aus dem Startstand: Jeder Fall ist ein Aufruf von pruefe(name, bestanden), am Ende steht Pruefung.zusammenfassung(). Ob der Konstruktor eine Exception wirft, prüfen Sie wie dort:

```
boolean geworfen = false;
try {
    Format.minuten("10:30");
} catch (NumberFormatException e) {
    geworfen = true;
}
pruefe("minuten(\"10:30\") wirft NumberFormatException", geworfen);
```

- (A) schreibt PruefungBuchung: jede Invariante des Konstruktors einmal verletzt, dazu eine gültige Buchung mit genau einer Person. Außerdem PruefungRaum: eine Buchung, die genau dann beginnt, wenn die vorige endet, gelingt; eine überlappende und eine mit zu vielen Personen werden abgelehnt; genau so viele Personen wie Plätze gelingen; die 21. Buchung auf einen vollen Raum wird abgelehnt.
- (B) schreibt PruefungZeitfenster: dauerMinuten; der Grenzfall „endet 10:00, nächste beginnt 10:00“ in beiden Richtungen gefragt; teilweise Überlappung; ein Fenster liegt ganz im anderen; gleiche Uhrzeit an einem anderen Tag; toString; jede Invariante des Konstruktors einmal verletzt; 07:00 bis 22:00 als erlaubter Grenzwert.

Findet ein Fall einen Fehler, repariert ihn die Person, der die Klasse gehört, in einem eigenen Commit, dessen Nachricht den Fall nennt. Am Ende enden alle Prüfprogramme mit 0 FEHLER.

Machen Sie danach gemeinsam eine Probe: Ändern Sie in istFrei testweise die Schleifenbedingung `i < anzahl` in `i < buchungen.length` und starten Sie PruefungRaum. In der Entwicklungsumgebung oder mit Maven beginnt die Meldung so:

```
Exception in thread "main" java.lang.NullPointerException: Cannot invoke
"freiraum.Buchung.getZeitfenster()" because "this.buchungen[i]" is null
```

Wer wie im README mit javac ohne -g übersetzt, liest `this.buchungen[<local2>]` statt `this.buchungen[i]`. Tragen Sie die Zeile aus Ihrem Lauf in NOTIZEN.md ein, erklären Sie in einem Satz, warum `buchungen[i]` dort null ist, und bauen Sie die Änderung zurück. Mergen Sie beide Branches nach main.

Sechste Aufgabe (30 Minuten): Erzeugen Sie gezielt einen Merge-Konflikt und lösen Sie ihn gemeinsam.

- Beide: `git switch main, git pull`.
- (A) ändert in `Zeitfenster.toString()` die return-Zeile in ein anderes Format, committet auf main und pusht.
- (B) holt vorher nichts, ändert dieselbe Zeile in ein drittes Format, committet und pusht. Git lehnt den Push ab:

```
! [rejected]          main -> main (fetch first)
error: failed to push some refs to '<remote>'
```

- (B) holt den Stand mit `git pull` (statt `<remote>` steht bei Ihnen die URL Ihres Repositories). Git meldet:

```
Auto-merging src/main/java/freiraum/Zeitfenster.java
CONFLICT (content): Merge conflict in src/main/java/freiraum/Zeitfenster.java
Automatic merge failed; fix conflicts and then commit the result.
```

- `git status` zeigt both modified: `src/main/java/freiraum/Zeitfenster.java`. Öffnen Sie die Datei zu zweit. Zwischen <<<<<< HEAD und ===== steht die Fassung aus dem eigenen Commit von B, zwischen ===== und >>>>>> die hereingeholte Fassung von A, hinter den Pfeilen die Commit-ID.

- Entscheiden Sie gemeinsam, welche Zeile bleibt. PrüfungZeitfenster und alle folgenden Startstände erwarten das Format Di 09:00–10:00. Löschen Sie alle drei Markerzeilen, starten Sie PrüfungZeitfenster, dann `git add src/main/java/freiraum/Zeitfenster.java`, `git commit -m "Konflikt in toString aufgelöst"` und `git push`. (A) holt den Stand mit `git pull`.

Bleiben Markerzeilen stehen, meldet der Compiler in der Zeile mit <<<<<< HEAD den Fehler `error: illegal start of type`.

Siebte Aufgabe (45 Minuten): Greifen Sie die Klassen der anderen Person an. Maßstab ist die Kriterienliste guter Kapselung aus T02B (Tafelfoto in Moodle); nach dieser Liste bewerten die Tutorinnen und Tutoren Ihr Blatt 1.

- Legen Sie im Paket `freiraum` eine Klasse `Sabotage` mit `main` an. Im selben Paket wie Ihre Klassen erreicht sie alles, was nicht `private` ist. Erzeugen Sie gültige Objekte.
- (A) greift `Buchung` an, (B) greift `Zeitfenster` an, beide dürfen `Raum` angreifen. Jeder Angriff besteht aus höchstens drei Zeilen und versucht, ein Objekt in einen Zustand zu bringen, den eine Invariante aus `NOTIZEN.md` verbietet: ein Ende vor dem Beginn, eine Buchung ohne Personen, zwei überschneidende Buchungen im selben Raum, ein Raum mit mehr als 20 Buchungen. Erlaubt ist alles, was kompiliert.
- Tragen Sie je Angriff unter „Sabotage“ in `NOTIZEN.md` ein: die drei Zeilen, das Ergebnis (kompiliert nicht, Exception mit ihrer ersten Zeile wörtlich, oder widersprüchlicher Zustand mit der Ausgabe), das verletzte Kriterium der Kapselungsliste und die Reparatur. Gelingt Ihnen kein Angriff, notieren Sie den stärksten Versuch und die Zeile, an der er scheitert.
- Die angegriffene Person repariert jeden gelungenen Angriff in ihrer Klasse und startet alle Prüfprogramme. Die Klasse `Sabotage` löschen Sie danach; ihre Zeilen stehen in `NOTIZEN.md`.

Mergen Sie alles nach `main`, setzen Sie den Tag und pushen Sie ihn:

```
git tag abgabe-1
git push origin abgabe-1
```

Packen Sie den Projektordner mit dem Ordner `.git` als ZIP und laden Sie ihn hoch.